

SIEMLab — Architecture & Setup

This document consolidates the design process behind the lab build. It focuses on **architecture, design rationale, and deployment sequence**. Detailed commands and configuration files live in a companion document, `02-Commands-and-Configuration.md`.

1. Project Goal & Context

A self-built detection engineering lab consisting of one Active Directory Domain Controller, one Windows 10 client, and one Splunk server, with Sysmon (SwiftOnSecurity configuration) forwarding endpoint telemetry into Splunk for analysis and detection development.

2. Design Direction & Rationale

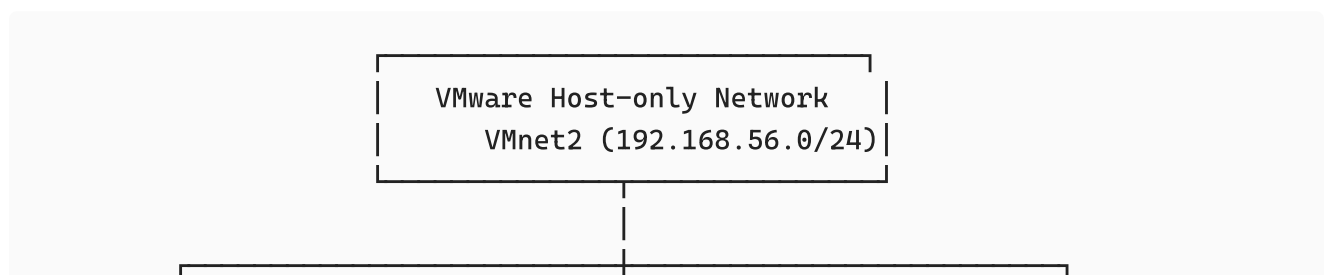
Constraint: Single laptop, 16 GB RAM, ~162 GB storage.

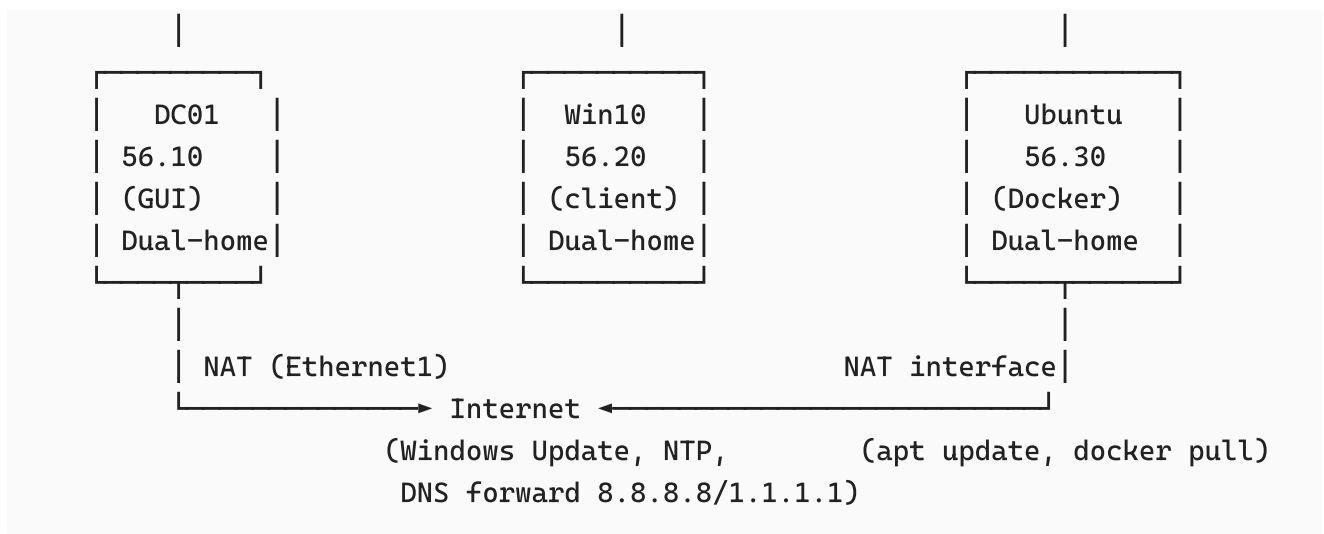
Approach chosen: Hybrid VMware + Docker.

Reasoning: Active Directory cannot be containerized, so the Windows components (DC, Win10 client) run as full VMs. The Linux-hosted service (Splunk) runs in Docker instead, which reduces resource overhead, builds additional Docker skills, and simplifies volume management, versioning, and future expansion (e.g., adding new containerized tools alongside Splunk).

Final decision: Windows components (DC, Win10 client) run as full VMs in VMware Workstation. Splunk runs via Docker Compose on a minimal, GUI-less Ubuntu Server VM. Portainer was added as a separate deployment (its own Compose file, not merged with Splunk's) since it manages the Docker engine as a whole and shouldn't be tied to Splunk's container lifecycle. Running both to gain hands-on Docker administration experience, including a GUI-based management layer.

3. Overall Architecture





Domain Controller (DC01) — Dual-homed: one Host-only interface (internal network) and one NAT interface (internet access). Roles: Active Directory Domain Services, internal DNS server, and DNS forwarder to the internet.

Windows 10 client — Dual-homed: one Host-only interface (internal network) and one NAT interface (for software installation), with DNS pointed entirely at the DC. Role: domain-joined workstation running Sysmon and the Splunk Universal Forwarder, and serving as the target host for Atomic Red Team simulations.

Ubuntu Server — Dual-homed: Host-only (internal communication) plus NAT (package/image downloads). Hosts Docker with two containers — Splunk (SIEM) and Portainer (Docker management) — and will later host Volatility 3 in a Python virtual environment for memory forensics.

Why Windows Server GUI (Desktop Experience) instead of Server Core for the DC

Server Core was the initial choice for resource efficiency, but Desktop Experience was selected instead. It still meets the lab's resource budget, allows direct administration through Server Manager, DNS Manager, and ADUC (Active Directory Users and Computers) without needing RSAT (Remote Server Administration Tools) installed on a separate management host, and better reflects the GUI-based Windows Server environments common in many production environments.

4. Resource Allocation

VM	RAM	vCPU	Disk (thin-provisioned)
DC (Windows Server GUI)	2 GB (may increase to 3 GB if headroom allows)	2 cores	40 GB

VM	RAM	vCPU	Disk (thin-provisioned)
Windows 10 client	2 GB	2 cores	60 GB
Ubuntu Server	4 GB (recommended minimum for Splunk)	2 cores	30 GB
Total	~10 GB / 16 GB		~130 GB / 162 GB max

Resource-saving principles applied:

- All disks are thin-provisioned (dynamically allocated), not fixed-size.
- Splunk's index size is capped (`maxTotalDataSizeMB`) to prevent unbounded disk growth over the lab's operational lifetime.

5. Network Architecture — Design Principles

5.1 Why an isolated network (Host-only, not Bridged/default NAT)

The lab runs Sysmon telemetry collection and adversary simulation (Atomic Red Team), so it needs to stay isolated from other devices on the home network to avoid both security risk and irrelevant log noise contaminating the dataset.

5.2 Gateway vs. DNS — two independent concepts, not to be conflated

This was an important early misconception worth documenting clearly:

- A **gateway** is only needed when traffic must leave the current subnet. Within the internal `192.168.56.0/24` subnet, hosts communicate directly over the virtual switch — **no gateway is required**.
- A **DNS server** resolves names and is a completely separate function from routing.
- **Result:** the Host-only interface on every VM has **no gateway configured**. Only the NAT interface has a gateway, assigned automatically via VMware's built-in NAT/DHCP service.

5.3 DNS Architecture: Client → DC → Forwarder

- All clients (Win10, Ubuntu) use a **single DNS server: the DC**.
- The DC acts as the **authoritative DNS server** for the internal AD zone, and also has a **forwarder** configured to `8.8.8.8 / 1.1.1.1` to resolve external domains.

- **Mechanism:** clients query the DC for everything. The DC determines internally whether a name belongs to its own zone (answers directly) or is external (forwards the query out and returns the result). Clients never need to know a forwarder exists — only one DNS address, ever.

5.4 Preventing the DC's NAT interface from polluting the AD DNS zone

On the DC's NAT interface, **Dynamic DNS Registration** must be disabled (uncheck "Register this connection's addresses in DNS" under Advanced TCP/IP settings). Without this, the NAT interface's IP can auto-register into the internal DNS zone, corrupting AD record lookups.

6. Deployment Sequence

1. **Provision three base VMs:** install ISOs for the DC (Windows Server 2019, GUI evaluation), Windows 10, and Ubuntu Server 24.04.
2. **Create a custom Host-only network** (VMnet2, 192.168.56.0/24) via the VMware Virtual Network Editor.
3. **Configure the DC:**
 - Assign static Host-only IP (192.168.56.10), no gateway.
 - Rename the host to `DC01` .
 - Install the AD DS role and promote to Domain Controller for the internal domain `cslab.local` .
 - Add a second (NAT) interface for dual-homing so the DC can reach the internet.
 - Configure a DNS forwarder to `8.8.8.8 / 1.1.1.1` for external resolution.
 - Disable Dynamic DNS Registration on the NAT interface.
 - Manually set DNS on both interfaces to point back to the DC itself.
4. **Configure the Windows 10 client:**
 - Assign static Host-only IP (192.168.56.20), DNS pointed at the DC, no gateway.
 - Rename the host to `Host-Victim` .
 - Join the `cslab.local` domain.
 - Add a second (NAT) interface for dual-homing.
5. **Configure the Ubuntu Server:**
 - Assign static Host-only IP (192.168.56.30) via Netplan, DNS pointed at the DC.
 - Set hostname to `cybersecservice` .
 - Add a second (NAT) interface via DHCP, with `use-dns: false` to preserve the DC as the sole DNS source.
 - Verify routing has exactly one default route, via the NAT interface.
6. **Install Docker on Ubuntu** and deploy Splunk via Docker Compose.

7. **Deploy Portainer** (separate Docker Compose file) for web-based container management.
 8. **Install Sysmon** (SwiftOnSecurity configuration) on the Windows 10 client.
 9. **Install the Splunk Universal Forwarder** on Windows 10, pointing the receiving indexer at Ubuntu (port 9997) and the deployment server at Ubuntu (port 8089).
 10. **Configure the Splunk Deployment Server** (instead of manually editing `inputs.conf` on the forwarder):
 - Create a deployment app (`sysmon_inputs`) containing `inputs.conf` on the Splunk server.
 - Create a server class, add the forwarder, and assign the app.
 - Verify successful check-in via the Forwarder Management UI.
 11. **Verify log flow** into Splunk.
 12. **Install Atomic Red Team** (Invoke-AtomicRedTeam) on Windows 10 to prepare for MITRE ATT&CK technique simulation, and verify corresponding detections appear in Splunk.
 13. **Install Volatility 3** (Python virtual environment on Ubuntu) in preparation for the memory forensics phase.
-

7. Key Lessons & Principles

- **Clear separation of roles:** Gateway $\neq$ DNS server $\neq$ Domain Controller. Even though this lab's DC handles both AD and DNS, these remain two independent functions configured separately and should not be conflated conceptually.
- **Container role separation:** Splunk (an application service) and Portainer (an infrastructure management tool) are kept in separate Compose files to avoid coupling their lifecycles during restarts or upgrades.
- Learned how to configure log forwarding from the Windows 10 client to the Splunk server.
- **Deployment Server discipline:** once a forwarder is managed by the Deployment Server, managed configuration files on that forwarder must never be edited by hand — any manual change is overwritten on the next check-in. The Deployment Server is the single source of truth.
- Learned how to troubleshoot issues encountered while testing log forwarding from the client to the Splunk server — checking everything from connectivity issues down to the forwarder's own Splunk log (`C:\Program Files\SplunkUniversalForwarder\var\log\splunk\splunkd.log`). In the process, discovered a new class of issue related to the Splunk Universal Forwarder's read permissions for certain special log sources, such as Sysmon logs and Windows Event Logs.

- **Least-privilege tooling setup:** when installing Atomic Red Team on the Win10 client, `Set-ExecutionPolicy` was scoped to `-Scope CurrentUser` rather than `LocalMachine` — the minimum change needed to run local/unsigned scripts, without loosening execution policy for the whole machine.
- **Environment isolation for new tooling:** Volatility 3 was installed on the Ubuntu host inside its own Python virtual environment (`~/tools/volatility-venv`), kept separate from system Python and other tooling, ahead of the Week 6 memory forensics phase.